

Unified Software Requirements Specification (SRS) & Project Plan

1. Document Information

Title	GRID (Graphs for Relational Information and Dependencies)
Team Number	16
Version	Version 0.1 Baseline

2. Personnel & Roles

Team Member	McMaster Email Address	Technical Disciplines
Kris Vuong	vuongk3@mcmaster.ca	Frontend, Data
Eunsu Kim	kime46@mcmaster.ca	Frontend, Testing
Oliver Li	li2177@mcmaster.ca	Backend
Hudson Sheng	shengh8@mcmaster.ca	Databases
Jinseok Jeon	jeonj10@mcmaster.ca	Project Manager
Gray Niederhuber	niederhg@mcmaster.ca	Frontend

3. Definitions, Acronyms & Abbreviations

AI - Artificial Intelligence
 API - Application Programming Interface
 DB - Database
 ERD - Entity-Relationship Diagram
 LLM - Large Language Model
 NFR - Non-Functional Requirement
 PR - Pull Request
 SRS - Software Requirements Specification

4. Contributions

Team Member	Authored sections	Reviewed sections	Contribution (%)
Gray Niederhuber	#7		
Jinseok Jeon	#9		

Hudson Sheng	#8	#9, #10, #11	20%
Oliver Li	#5, 6, 11	#8	18%
Eunsu Kim	#10	#8, #5	15%
Kris Vuong	#12-17	#5, #8, #9	20%

5. Executive Summary & Purpose

Many project management applications on the market, such as JIRA and Linear, focus on capturing individual tasks to represent project progress. However, tracking the completion of individual tasks alone does not adequately answer several critical questions: which sections of a project are lagging, how tasks relate to one another at different levels of abstraction, and what the overall progress of the project looks like. Without a clear view of these relationships, project managers may struggle to identify bottlenecks, coordinate parallel work, and allocate resources effectively.

This web application targets project managers and members of cross-functional teams. It will act as a general-purpose, network graph-based project management tool designed to provide a more comprehensive view of project structure, dependencies, and progress.

The application will provide structured inputs for team members to contribute project information throughout the development lifecycle, including meeting notes, requirements, derived features, tasks, logs, designs, and other relevant artifacts. Centralizing this information in a structured format will improve its traceability and maintainability while establishing a consistent source of project information.

Using this information, the application will provide a dynamic network graph that visualizes tasks, dependencies, parent-child relationships, blockers, and other key project information. Team members and project managers will be able to explore project progress at different levels of detail, allowing them to focus on the information most relevant to their roles. For example, a finance director may choose to view only high-level finance-related tasks that remain unfinished, while a project manager may examine dependencies and bottlenecks across multiple sectors. The graph will dynamically reflect changes to the underlying project information, providing an up-to-date view of progress as work evolves.

The application will also generate concise AI summaries of project progress, highlighting in-progress tasks and major bottlenecks for quick review.

Ultimately, the project aims to improve operational efficiency and communication clarity by helping project managers identify delays and coordinate work more effectively, while enabling team members to better understand the status and dependencies of their own work in relation to the broader project.

Part II: Software Requirement Specifications

6. Stakeholders & User Personas

Primary client: Project manager, stakeholder viewers

Archetype: coordinating among **cross-functional** teams, ensuring the progress of a complex project of **any** nature. A visualized project will help them abstractly grasp the progress of the project without getting sidetracked by the detailed updates.

Expectation: the visualization offers intuitive interaction functionalities that expose the progress of the project, the potential dependencies/bottlenecks, and the level of detail of individual task groups. The map I view at any given time is guaranteed to illustrate the latest vision of the project.

Example: a big ensemble band is planning to embark on an international performance. The band director needs to ensure various aspects of the project are progressing as planned; that may include the availability of the musicians, the logistics of traveling, the funding of the tour, the music preparation, and so on.

Secondary client: Project members

Archetype: ground-level task forces, keeping artifacts **updated** and **complete**. This includes updating meeting minutes, task logs, status and details of the tasks when available, such that the project map reflects the most updated vision of the project.

Expectation: each type of artifact (meeting notes, tasks, logs...) is templated with an intuitive, quick input process. My input will be pushed to the project map instantaneously.

Example: a sponsorship coordinator is responsible for **maintaining** the artifacts related to sponsorship, like the meetings with potential sponsors, or the status of funding applications.

7. Constraints & Assumptions

Server Costs

- If we choose to rely on an open-source LLM for automated system features such as document generation and data processing, there will be server costs associated with hosting the model on a dedicated server
- There will also be server costs involved with self-hosting our website and running the backend
- If we choose to additionally self-host our own database, this will further add to the server costs

Token Costs

- If we choose to rely on API calls to an external LLM service for automated system features such as document generation and data processing, we will have to consider token costs involved with using the API

Infrastructure Costs

- As previously mentioned, there will be server costs associated with all parts of the website/application we choose to self-host, which can include frontend, backend, open source LLM, and database
- There will be costs involved with registering a domain to host our website on
- There may be additional cloud infrastructure service costs should we choose to rely on a third-party service for database hosting

Hardware Constraints

- A major selling point we are aiming for is responsiveness & ease of use, but with some relatively complex behaviours and ui components. We will need to use intelligent software design to ensure that the system is consistently responsive, reactive, and quick to load in the user's browser regardless of browser choice, hardware, and OS.

Privacy Concerns

- This app will be hosting confidential user and project data that belongs to clients, so security will be important. It will face all standard security risks that pose threats to websites and web apps, including SQL injection, cross-site scripting, cross-site request forgery, distributed denial-of-service attacks, etc.
- We will also be required to follow standards for handling this confidential data, including PIPEDA, GDPR, LGPD, and PDPA. If we choose to also handle credit card payments for premium features, etc., we will need to follow PCI DSS
- We will need to use standard security practices including encryption, input validation, etc in order to securely store user data

8. Functional Requirements

P0 Requirements
<u>P0.1 - Task-Dependency Visualization</u> Description: The user will be able to view the current project constructed with all the tasks and dependencies. Inputs: - None Expected Output: - The data will be visualized in the form of a dependency relationship graph Data Flow: Database → Backend → Frontend Focus: Frontend <u>P0.2 - Graph Navigation</u> Description: The user is able to expand and collapse sections relating to tasks and relations. While focused on a specific section of tasks, the user is able to navigate to other sections by panning around to view nearby tasks. Inputs: - Scroll up/down - Mouse click and drag Expected Output: - The graph zooms in or out according to the user's scroll input.

- The graph pans horizontally and vertically according to the user's mouse movement.

Data Flow: User → Frontend → Backend → Frontend

P0.3 - Graph Manipulation

Description: The user is able to add, delete, and modify tasks and dependencies represented as nodes and directed edges

Inputs:

- Selecting to add or delete a task
- Selecting two tasks and creating dependency relationship

Expected Output:

- Database updates based on relationship changes

Data Flow: User → Frontend → Backend → Database

Focus: Backend

P0.4 - Manual Task Update

Description: The user is able to click into individual tasks and edit every component of it, such as title, description, deadlines, task status, etc.

Input:

- Entering text into input fields
- Selecting an option from a dropdown
- Selecting an option from a calendar view

Expected Output:

- Database updates based on modification to task

Data Flow: User → Frontend → Backend → Database

Focus: Backend

P0.5 - Graph Visualization Synchronization

Description: The user is always shown the most up to date version of the data represented in a graph. Whenever a task or dependency is created, deleted, or modified, the application will construct a new graph to be displayed.

Inputs:

- Update to the project data

Expected Output:

- New graph is constructed and displayed to all users

Data Flow: Database → Backend → Frontend

Focus: Frontend

P1 Requirements

P1.1 - Automatic Data Processing

Description: The user is able to upload various files containing new or updated information relevant to the project. The app will automatically process the information using an LLM in order to modify tasks and dependencies based on the information given. The changes will then

be presented to the user to verify and apply, giving them the opportunity to add or remove changes based on relevance. The file should be stored in case a change needs to be referenced further down the line.

Input:

- PDF documents
- Audio recordings
- Text documents

Expected Output:

- File stored in database
- Prompt to validate modifications to the project

Data Flow: User → Frontend → Backend → LLM Call → Database

Focus: Backend

P1.2 - Automatic Progress Report Generation

Description: The user will receive a general update notification whenever a task they are assigned to changes description, deadline, status, or progress. This can come in the form of either an in-app notification or email. The user is also able to push a progress report to all members of the team that contains the current up to date summaries of all in-progress tasks.

Input:

- Button click to generate progress report
- Change in task information

Expected Output:

- Corresponding users will receive a summarized notification about their respective task progress

Data Flow: User → Frontend → Backend → Database → Other Users

Focus: Backend

P1.3 - User Account Management

Description: The user is able to create an account with a username and password.

Input:

- Username
- Password

Expected Output:

- Account is created in the database
- User is able to log in using account credentials

Data Flow: User → Frontend → Backend → Database

Focus: Backend

P1.4 - Multi Project Management

Description: The user is able to maintain more than one project at a time. A single user is able to create projects that are tied to the account and is able to modify them.

Input:

- User chooses to create new project

Expected Output:

- New project is created on the database that is associated with the user who created it

Data Flow: User → Frontend → Backend → Database

Focus: Backend

P1.5 - Project Access Control

Description: The user is able to add other users as collaborators to each project directory. Each added user can be assigned privileges to be able to edit or read only.

Input:

- Usernames of collaborators
- Privilege selection

Expected Output:

- Added users gets a notification to accept the invitation to the project
- User information and permissions are updated in the backend in relation to the project

Data Flow: User → Frontend → Backend → Database → Invited User

Focus: Backend

P2 Requirements

P2.1 - Automatic Task Completion

Description: The system is able to automatically update task statuses based on changes to related dependencies. When a parent task is initiated, any child tasks required for the parent task that are currently incomplete will create a pop up prompt to the user to update the status to complete.

Input:

- Change in status of task with child dependencies

Expected Output:

- User is prompted to complete child task

Data Flow: User → Frontend → Backend → Database

Focus: Backend

P2.2 - Viewable Link Generation

Description: The user is able to generate a link that allows anyone to view the current graph without user authentication or editing privilege.

Input:

- User chooses to generate shareable link

Expected Output:

- Database stores a snapshot of the current project
- Backend creates a link to access the snapshot

Data Flow: User → Frontend → Backend → Database → Backend

Focus: Backend

P3 Requirements

P3.1 - Version History

Description: The user is able to view the changes made to individual tasks, as well as the changes made to the overall project.

Input:

- Selection of version change

Expected Output:

- Visual graph will show the historical versions of the project

Data Flow: User → Frontend → Backend → Database → Backend → Frontend

Focus: Backend

P3.2 - Version Control

Description: The user is able to roll back the project to a previous state before a certain modification. Each update to the project will be saved to the database as an edit on top of the initial task creation, allowing the user to roll back changes to a previous state without the need to store every version of the changes in its entirety.

Input:

- Selection of older version to roll back

Expected Output:

- Database removes all updates made after the selected version

Data Flow: User → Frontend → Backend → Database

Focus: Backend

P3.3 - Live Modification

Description: Multiple users are able to modify the document at the same time. Each time a modification is saved, the graph visualization for all current users updates in real time to accommodate the changes made.

Input:

- Editing user makes a change to a task/dependency

Expected Output:

- All active users' webpages update with new information

Data Flow: Editing User → Frontend → Backend → Database → Viewing User's Frontend

Focus: Frontend

P3.4 - Third Party Authentication

Description: The user is able to log in with a third-party account, like a Google or Microsoft account, through OAuth

Input:

- Log in through third-party account services

Expected Output:

- New user account created with data linked to third-party account

Data Flow: User → Frontend → Backend → Database

Focus: Backend

9. Data Strategy & Quantitative Metrics

Data Strategy:

- Data Type: Synthetic task/dependency graph data and publicly available meeting transcripts/project documents
 - LLM Benchmark Data: MeetingBank, MISeD, Meeting notes, requirement/project documentation, task updates
 - Ground Truth: Manually labelled tasks and dependencies for precision, recall, and F1-score evaluation
 - System Test Data: Up to 100 tasks and 200 dependencies
 - Source: Public datasets and internally generated by the team
 - Licensing: External dataset licenses will be reviewed before use
 - Privacy: Synthetic and publicly available non-confidential data only
 - Purpose: Evaluate LLM extraction accuracy, hallucination rates, and system performance
 - LLM Guardrails: Prompt constraints and output validation to reduce hallucinations and improve extraction reliability
- The team will manually generate and label ground-truth data, expected tasks, and dependencies. It will be used to evaluate the accuracy of the system.
 - Test datasets will have different sizes and complexity of tasks/dependencies to evaluate system performance and scalability.

Quantitative Metrics:

Component	Metric	Target Values
LLM Task Extraction	Precision	$\geq 90\%$
LLM Task Extraction	Recall	$\geq 90\%$
LLM Task Extraction	F1-score	$\geq 90\%$
Dependency Extraction	F1-score	$\geq 85\%$
Task Extraction	Duplication Rate	$\leq 2\%$
Extraction Pipeline	Processing Latency	$\leq 15s$ per 5-page text-based PDF
Backend Task CRUD (Backend CRUD being processed through the frontend)	Response Latency	$\leq 500ms$
Graph Visualization	Initial Rendering Time	$\leq 2s$
Graph Manipulation (drag/zoom/pan)	Interaction Latency	$\leq 100ms$

Summary Generation	Generation Time	$\leq 10s$
System Reliability	Successful Request	$\geq 99\%$

10. Non-Functional Requirements (NFRs)

Usability & Human-Computer Interaction

- The interface will display the information on projects such as tasks, requirements, dependencies, and progress in a consistent manner to PMs and their teams.
- The interface controls for adding, viewing and editing tasks and requirements will be clearly presented and easy to access.
- The system shall provide feedback after user actions to indicate either the successful execution or the unsuccessful execution with clear error messages for failure.
- The software will be able to run on some of the most popular web browsers such as Google Chrome, Microsoft Edge, and Safari and can function independently of the desktop operating system used.
- Some of the web accessibility features, such as having readable text, high contrast, keyboard navigation, navigation options, and differentiating between tasks, requirements, and their interdependencies without relying on colours, will be included in the interface design.

Performance, Scalability & Speed

- The dependency graph should continue to respond to a project that has up to 100 tasks and 200 dependencies.
- The application loading time should meet the performance target defined in the Quantitative Metrics section.
- Normal backend operations like creation or updating of tasks and requirements must satisfy the latency response goal set out in the Quantitative Metrics.
- If Live Modification is used, changes made by one user should appear for other active members without requiring them to restart the application.

Security, Authentication & Data Privacy

- Users will be authenticated before being permitted to access or modify project information.
- Permissions will also be verified on the backend side and will not solely depend on frontend limitations.
- If access control is enabled for the project, then the user can do only those things that they are permitted to do.
- Passwords will be stored in hashed form without any plaintext, while API keys and database credentials will be stored in environment variables or something similar to that, away from the source code.
- The communication between client and server applications will be established through an encrypted connection (HTTPS/TLS).
- User input data will be checked at the backend level before being processed or stored to reduce invalid input and database attacks such as SQL injection.

- If an external LLM service is used, only necessary data will be shared with the service, and only if the project's privacy conditions are met.

Reliability, Fault Tolerance & Maintainability

- In case of an unsuccessful update, the data should not be left partially modified. The previously valid data should not be impacted by this issue at all.
- The system is expected to be able to handle faulty data input, failed server or third-party application requests, and network problems, displaying the correct error message to the user.
- The system will keep referential integrity in place for all related project items, which include tasks, requirements, and dependencies.
- Modifications of one project item will not cause the creation of inconsistent and invalid dependencies between other items in the project.
- In case change history is implemented, then modifications of significant project items will be traceable, and users will know what has been changed and when the modification happened.
- The system should be easy to maintain, so changes to one main component should not require changes to unrelated parts of the program.

11. Risk Management & Feasibility Analysis

Likelihood \ Impact	Neglegible	Moderate	Severe
Very Likely		F	C
Possible	E	D	
Very Unlikely			B, A

Risk list and their mitigations

- A. Confidential user data breach
 - a. Encryption mechanism
- B. Loss of user data
 - a. The database resides on a reliable server (e.g., CAS server).
- C. Core function missing
 - a. Members will consult with subject matter experts to accelerate the research and development process.
- D. Users turn away from using this application and stick to their existing project management tool
 - a. Sticking to hard and soft requirements defined above.
- E. Low complexity as a capstone project
 - a. Focus on user-oriented software design. Well-made software is great software
- F. High complexity from various intricate components, but the application itself fails to connect with the user (fancy and abundance of features don't imply that users will like it)
 - a. Incremental development

Part III: Project Development & Execution Plan

12. Team Communication & Meeting Cadence

The team will meet internally **once per week, on Mondays at 1:30-2:20pm, in-person**. More frequent meetings may be organized as needed, determined by team consensus. The team commits to consistent daily virtual communication via Discord group chat.

All meeting records are maintained on the project GitHub repository at https://github.com/AG-GROUP-LIMITED/COMPSCI_4ZP6_CAPSTONE/tree/main/meetings.

For the Fall 2026 semester, the team will check in with the supervisor (professor) on **October 26, 2026** and **November 9, 2026**, and with the TA on **October 8, 2026**. For the Winter 2027 semester, the team plans to again meet with the professor twice, and the TA once.

13. Engineering Workflow & Version Control

The repository is located at https://github.com/AG-GROUP-LIMITED/COMPSCI_4ZP6_CAPSTONE, and follows the structure below:

- `/src/` contains all source code and tests for the deliverable software project
- `/docs/` contains all written deliverable submissions
- `/meetings/` contains the team's meeting records

The team will be following the **GitFlow** branching model, described by the core branches:

- `main`: holds stable, tested, production-ready code
 - Code should never be committed directly to `main`. This will be enforced using branch protection rules.
- `develop`: acts as the integration branch for ongoing work and new features.
 - This branch is periodically merged into `main` (in lieu of a release branch)
- `feature_*`: short-lived branch off of `develop` to build features and merge back

Pull request (PR) review policy:

1. PRs into `main` **must** receive an approving review from **all 6** team members.
 - a. This will be enforced using branch protection rules.
2. PRs into `develop` must receive approving reviews from **at least 2** team members.
3. Team members shall send PR links in the Discord channel `#pr-review`.
 - a. Within a reasonable time frame (<1 day), other team members shall thoroughly review the proposed changes, leave comments if needed, and then approve.

CI/CD pipeline integration:

- GitHub Actions will be used to run checks on PRs, blocking the merge if checks fail.
- Automated checks will include linters (ESLint, Ruff), unit tests (React Testing Library, pytest, HTTPX), and build tests.

Issue tracking method: The team will use GitHub Issues to plan and track work items via a GitHub Kanban board: <https://github.com/orgs/AG-GROUP-LIMITED/projects/1>.

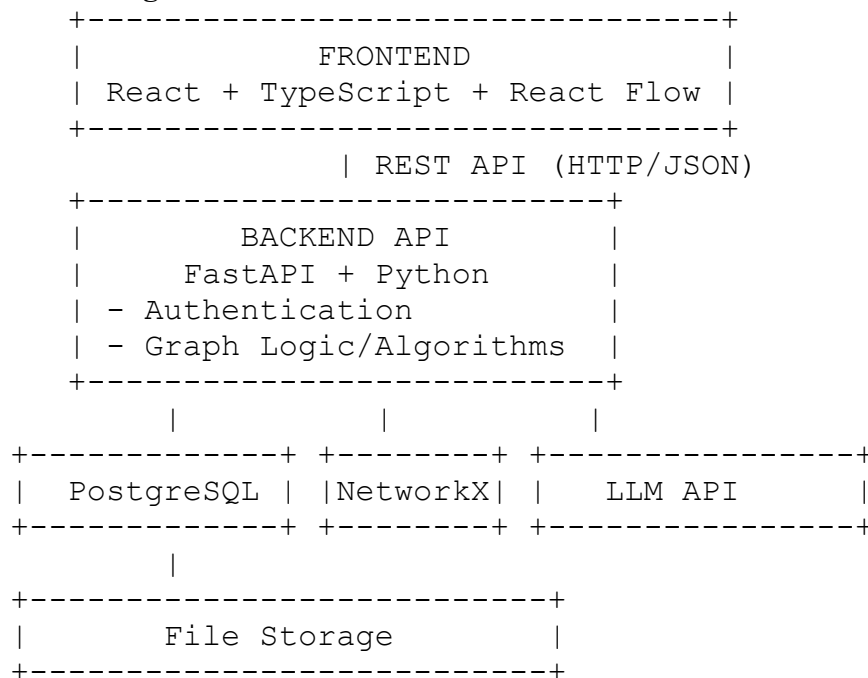
Agile/Scrum framework adoption, sprint length, backlog management: The team adopts Agile methodologies. At the weekly meeting, the team will give high-level updates on their current work. As needed, backlog items will be created, prioritized, assigned, and implemented. The team will use 2-week sprints.

14. Infrastructure & Compute Resources

As the project is a full-stack web application, the team will need to use CAS virtual servers to host the frontend, backend service, and the SQLite files/PostgreSQL database. Furthermore, the team plans to self-host an open-source LLM; GPU clusters are highly preferred; otherwise, the model can be hosted via CPU inference, but note that this will be a significant bottleneck.

15. Technology Stack & Architectural Rationale

Architecture Diagram:



Rationale:

- **Python (Backend):** Chosen for its readability, rapid development (high familiarity within the team), and mature ecosystem for graph algorithms and LLM integration.
- **FastAPI:** A lightweight Python framework with automatic request validation and API documentation, well suited to serving a separate frontend.
- **NetworkX:** Python library that provides a graph algorithm interface for dependency validation, cycle detection, and identifying blocked tasks.

- **TypeScript (Frontend)**: Provides static typing on top of JavaScript, improving maintainability and reducing type errors during development.
- **React + React Flow**: React supports reusable UI components, while React Flow provides interactive graph visualization and manipulation.
- **LLM API**: Extracts candidate tasks, dependencies, and project information from user-uploaded artifacts for manual review.
- **React Testing Library (Testing)**: Tests frontend components and user interactions.
- **pytest + HTTPX (Testing)**: Tests backend logic, API endpoints, and request validation, with integration tests verifying database interactions.
- **ESLint + Ruff (Linting)**: ESLint identifies code quality issues in TypeScript; Ruff checks common errors and style violations in Python, to ensure quality committed code.
- **GitHub Actions**: Automates testing, linting, type checking, and deployment to maintain code quality and catch integration issues early.

16. Proof of Concept (PoC) Demonstration Plan/Roadmap

Oct 12-18 (midterm recess):

- Skills development: familiarize self with chosen technology stack
- Build the boilerplate code

Oct 19-25: Design the data strategy, UI, API specification, backend architecture

- Design data structures
 - Define entities and their structure, ex. user, project, task, dependency, etc.
 - Define tables and relationships, ex. keys, columns, data types, nullable, etc.
- Design UI prototype
 - Mock UI pages, ex. login, dashboard, graph, task details, task editing, etc.
 - Define intended user workflows/actions
 - Develop a style guide, ex. colour, typography, components, interactivity, etc.
 - Define graph interactions, ex. zoom, drag, filter, expand, collapse, select, etc.
- Design backend architecture
 - API Endpoints: define the necessary endpoints and the respective request/response formats for APIs between backend/frontend/DB/LLM
 - Logic: determine the functions needed for the backend design, ex: authentication,, cycle detection, get tasks with no blockers, etc.
 - Miscellaneous: ex. rate limiting, error handling, etc.

Oct 26-Nov 1: Environment setup and integration MVP

- Environment setup: all team members can run the source code locally on their machine
- Initialize the frontend
 - Basic static page navigation/routing
 - Develop re-usable UI components and style guide
 - Render basic graph using mock data
- Initialize the backend
 - Set up FastAPI application
 - Set up DB connection (SQLite/Postgres)
 - Implement a basic API CRUD endpoint
- Initialize the database
 - Set up the DB (SQLite/Postgres)

- Create initial tables and relationships with mock data
- Begin frontend/backend integration
 - Be able to render data from the database, persist edits into database
- Set up test suite infrastructure
 - Simple frontend/backend component/unit tests
- CI/CD (GitHub Actions)
 - Set up branch protection rules and automated code checks on PR creation

Nov 2-8: Implement P0s

- Build the frontend features for P0.1-P0.5
- Build the backend features for P0.1-P0.5
 - Consider request validation, error handling
- Work on database operations for P0.1-P0.5
 - Implement how CRUD operations get inserted/updated to DB
- Build test suite as features get implemented, integrate with GitHub Actions (CI/CD)

Nov 9-15: Implement P0s + Integration

- Continue working on frontend P0s
 - Consider loading indicators, smooth page navigation, UI consistency
- Continue working on backend P0s
- Address load times, latency, errors, integration issues
- Continue integrating test suite with GitHub Actions (CI/CD)

Nov 16-21: Prepare PoC video

- P0s should be finished implementation with MVP features:
 - Frontend: Given sample data in the database, there is a dynamic graph visualization, dependency/node manipulation, and manual task update.
 - Backend: There are well-designed data structures for tasks/dependencies, and functions/endpoints are implemented for core functionalities.

Nov 22: Submit PoC video

17. Project Schedule & Gantt Chart

